CODESTREET 2026 · ROUND 1 IDEA SUBMISSION

THEME — CROSS-CHANNEL JOURNEY STITCHING

Cross-Channel Journey Stitching with Late-Binding Identity and a Bitemporal Journey Store

A journey store that remembers what it used to believe.

TEAM

Larpers

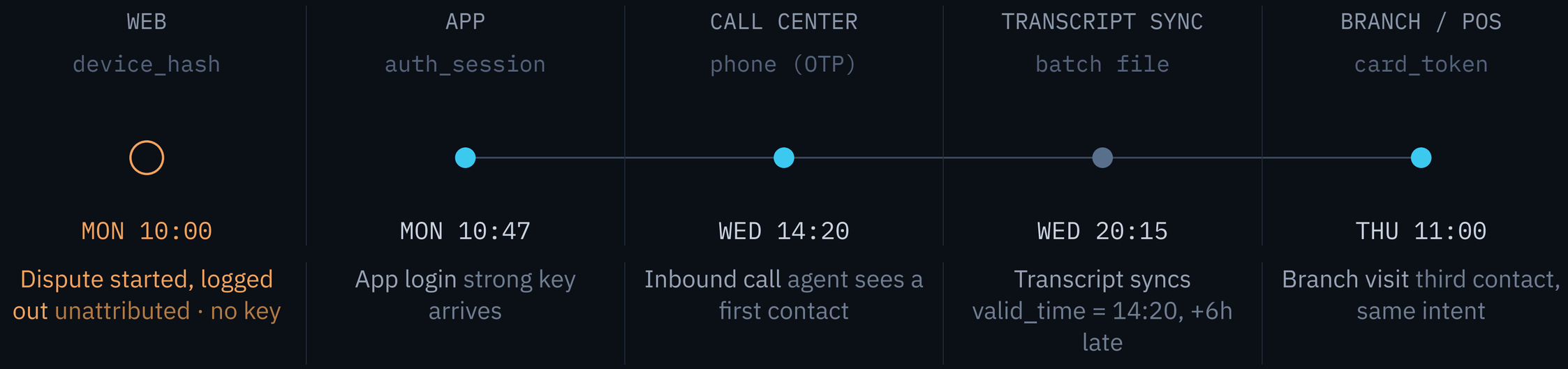
MEMBERS

Prathamesh | Ayush | Ved

valid_time · system_time

Five Touchpoints, Four Silos, No Shared Identity

One Card Member, one dispute. Each system holds a fragment keyed on its own identifier, and nothing joins them.



The 10:00 event belongs to nobody. It floats.

The transcript arrives after the decision it should have informed.

Nothing records what the system knew at 14:20.

THE FIELD

Everyone treats this as a join problem.

Collect the events, key them, draw one timeline.

OUR CLAIM

It is a state problem over an unreliable stream.

The join keys do not exist yet, the stream is out of order, and the answers decay.

identity binds late

The 10:00 session has no key until 10:47.

events arrive out of order

Batch syncs split valid_time from ingest_time by hours.

conclusions go stale silently

Scores computed pre-10:47 are never revisited.

Servicing cannot answer “Why didn't the agent see it?” — because no system stores past belief states.

Four Mechanisms, One per Failure

Each is a known technique from stream processing or record linkage. The novelty is the composition and the domain.

01

Late-Binding Identity with Retractions

Orphan events wait in a pending attribution buffer; a strong key retroactively claims them and downstream gets a retraction pair, never a silent overwrite.

`fixes: identity binds late`

02

Bitemporal Journey Store

valid_time and system_time as independent axes, so an AS-OF query reconstructs the exact view any agent had at any past instant.

`fixes: out-of-order arrival`

03

Intent Threading

Online clustering over referenced entity, intent similarity and a decaying time kernel separates concurrent intents into distinct threads.

`fixes: false repeat contacts`

04

Lineage-Tracked Invalidation

Derived artifacts store the input event IDs they came from, so a revision invalidates exactly the affected subgraph — and nothing else.

`fixes: stale conclusions`

Identity Binds Late

IDENTITY_STATE — EVERY EVENT CARRIES ONE

- **resolved**
  Strong key: card token, post-OTP phone, auth session, POS txn ID.
- **provisional**
  Splink match 0.70–0.95, stored with confidence and per-field evidence.
- **orphan**
  Below 0.70. Unattributed — kept, never discarded.

pending attribution buffer

`device_hash · ip/asn · phone_tail · time_bucket`

Indexed on weak signals. Bounded by event-time watermarks and allowed lateness.

THE BINDING SEQUENCE

MON 10:00 — Web dispute, logged out → Splink 0.61 → orphan

`buffered on device_hash + ip · timeline empty`

MON 10:47 — Authenticated app session → strong key → resolved

`binding event fires · scans buffer · claims the 10:00 orphan`

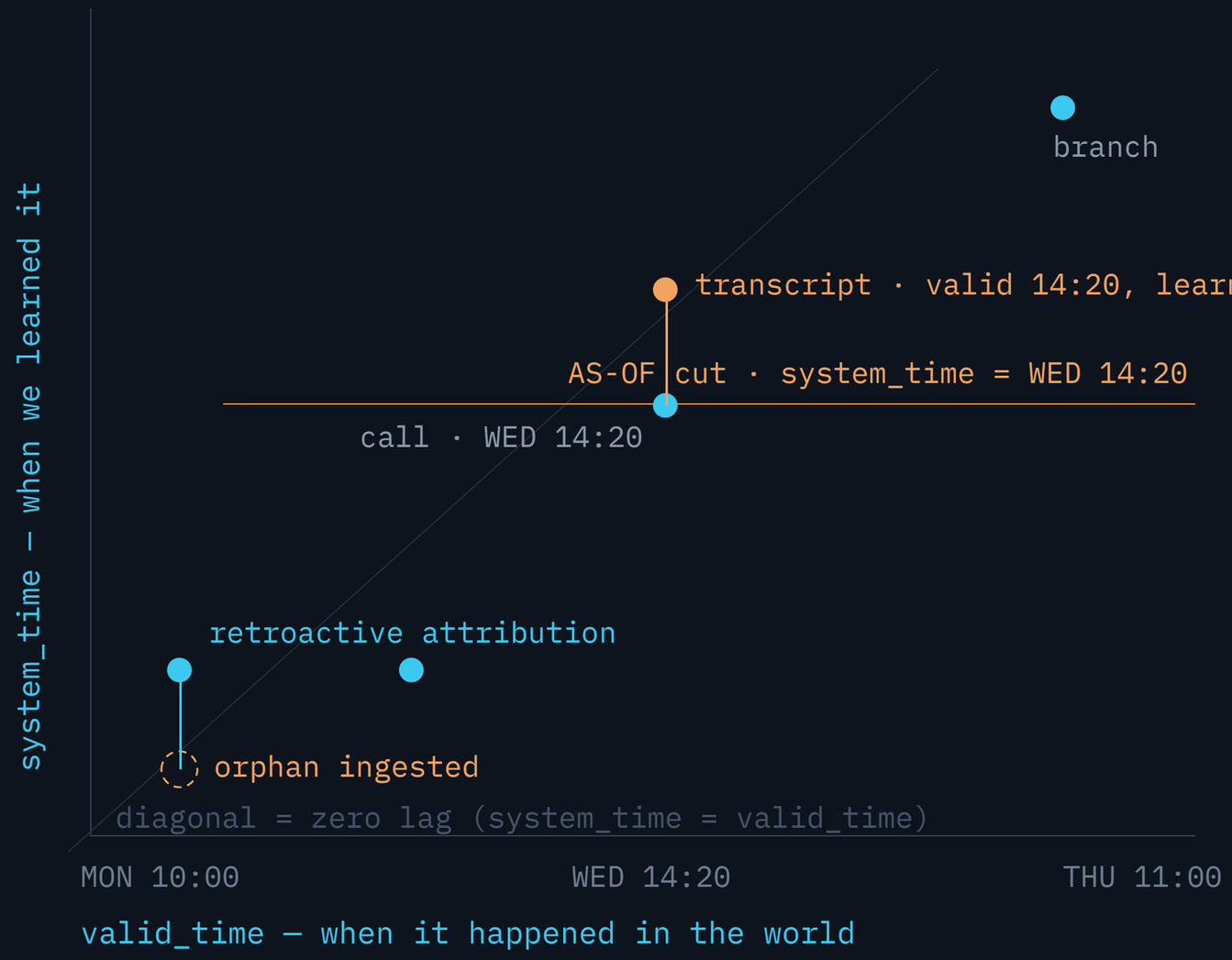
downstream — Consumers receive a retraction pair, not a silent overwrite

```
(−) event@E7731 · identity_state=orphan
(+) event@E7731 · customer=C1041 · system_time=10:47
```

A negative cancels the old assertion, a positive states the new one. Every consumer converges without replaying history.

Precedent: Flink and Kafka Streams changelog semantics — retract-then-assert is how mature stream processors already handle revision.

Two Clocks, Not One



AS-OF query

Reconstructs exactly what the agent saw at WED 14:20 — orphan detached, transcript absent, friction score lower.

knowledge_gap at contact time

The delta between the AS-OF view and what is known now. Not a narrative — a measurable quantity, per contact.

IMPLEMENTATION

SQL:2011 system-versioned temporal tables in PostgreSQL 16 — `tstzrange` + `btree_gist` exclusion constraints, versioned union-find for the identity graph. Production path: XTDB.

The technique is not the invention. The domain is.

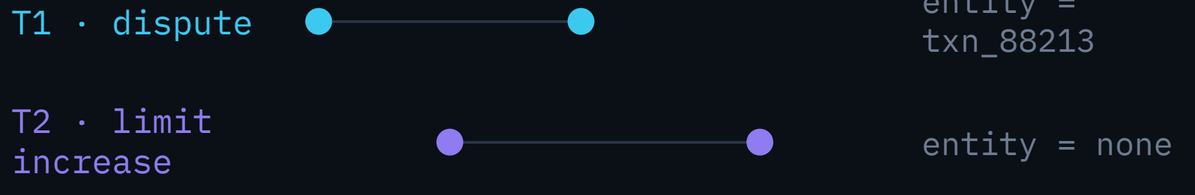
Concurrent Intents Must Be Threaded, Not Merged

SINGLE TIMELINE — TWO INTENTS TANGLED



repeat_contact = 4 · the second call falsely reads as an unresolved repeat

THREADED — TWO INDEPENDENT INTENTS



repeat_contact = 2 on T1 · T2 is a first contact, not a failure

CLUSTERING SIGNALS — ONLINE, WEIGHTED

referenced_entity — near-deterministic

Shared txn ID, case number, disputed amount.

intent_similarity — medium

Sentence-transformer embeddings over event text (all-MiniLM-L6-v2).

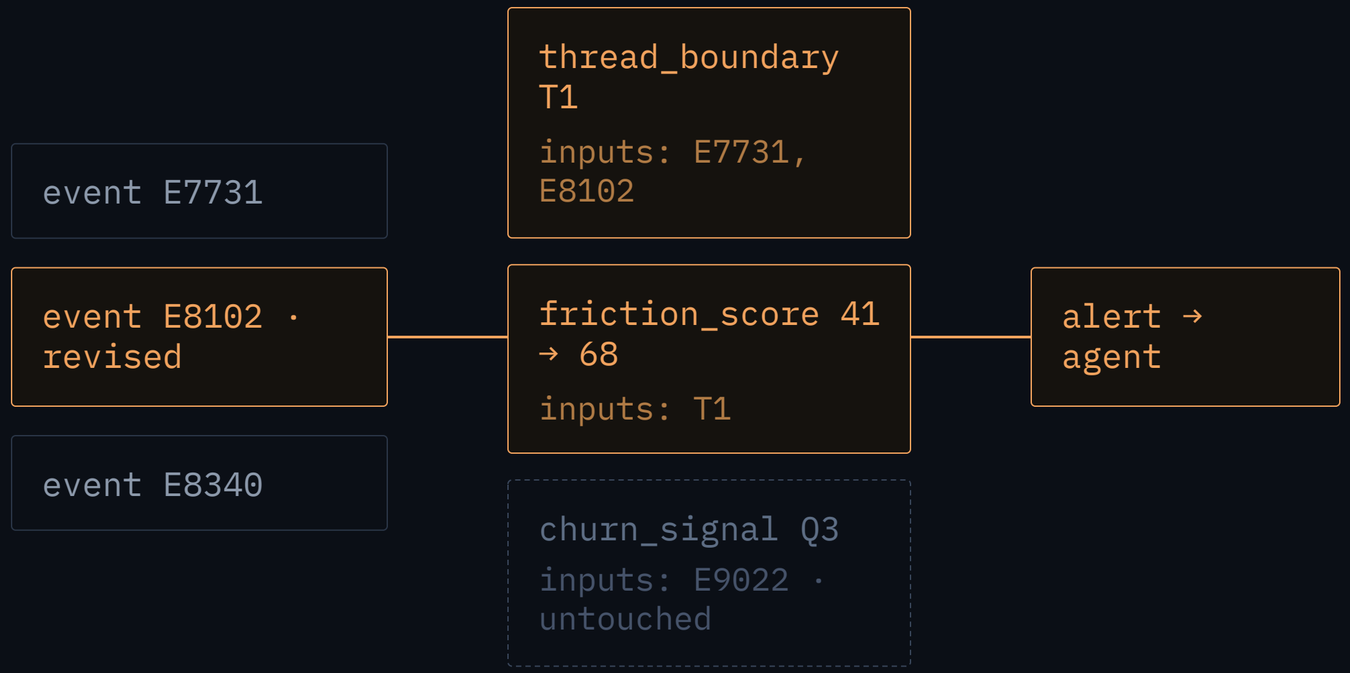
temporal_proximity — weak

Decaying kernel — 2 hours binds harder than 2 weeks.

Lineage: NLP conversation disentanglement (Elsner & Charniak; Kummerfeld et al.), applied to servicing events instead of chat turns.

Revisions Invalidate Exactly What They Touch

LINEAGE GRAPH — DERIVED ARTIFACTS STORE INPUT EVENT IDS



A revision walks the lineage graph and invalidates only the artifacts that consumed the changed event. Targeted subgraph invalidation, on the Differential Dataflow / Materialize model — not a nightly full rebuild.

HONEST EDGE CASE

An alert already sent cannot be unsent.

Internal state converges; a delivered side effect does not. The system emits a correction record and tracks `post_revision_alert_precision` as a first-class metric rather than pretending revision is free.

Six Layers, One Feedback Loop

producers: app · web · call-center · branch/POS

[1] INGEST

Redpanda topics

CustomerEvent { event_id, channel, valid_time, ingest_time, identifiers{}, payload }

[2] NORMALISATION

Shared intent taxonomy

canonical envelope · PII tokenised (HMAC + salt) pre-matching

[3] IDENTITY RESOLUTION

A · hash join on strong keys → resolved

B · Splink probabilistic (Fellegi-Sunter)

≥0.95 resolved · 0.70–0.95 provisional · <0.70 orphan

PENDING ATTRIBUTION BUFFER

device_hash · ip/asn · phone_tail · time_bucket

strong key → BINDING EVENT → retraction pair

The buffer feeds back into resolution — past state is revisable, not append-only.

[4] BITEMPORAL STORE

PostgreSQL 16

tstzrange + btree_gist · versioned union-find identity graph

[5] THREADING & SCORING

Threads, friction, lineage

entity + embedding + time kernel · repeat contact, channel-switch velocity

[6] SERVING

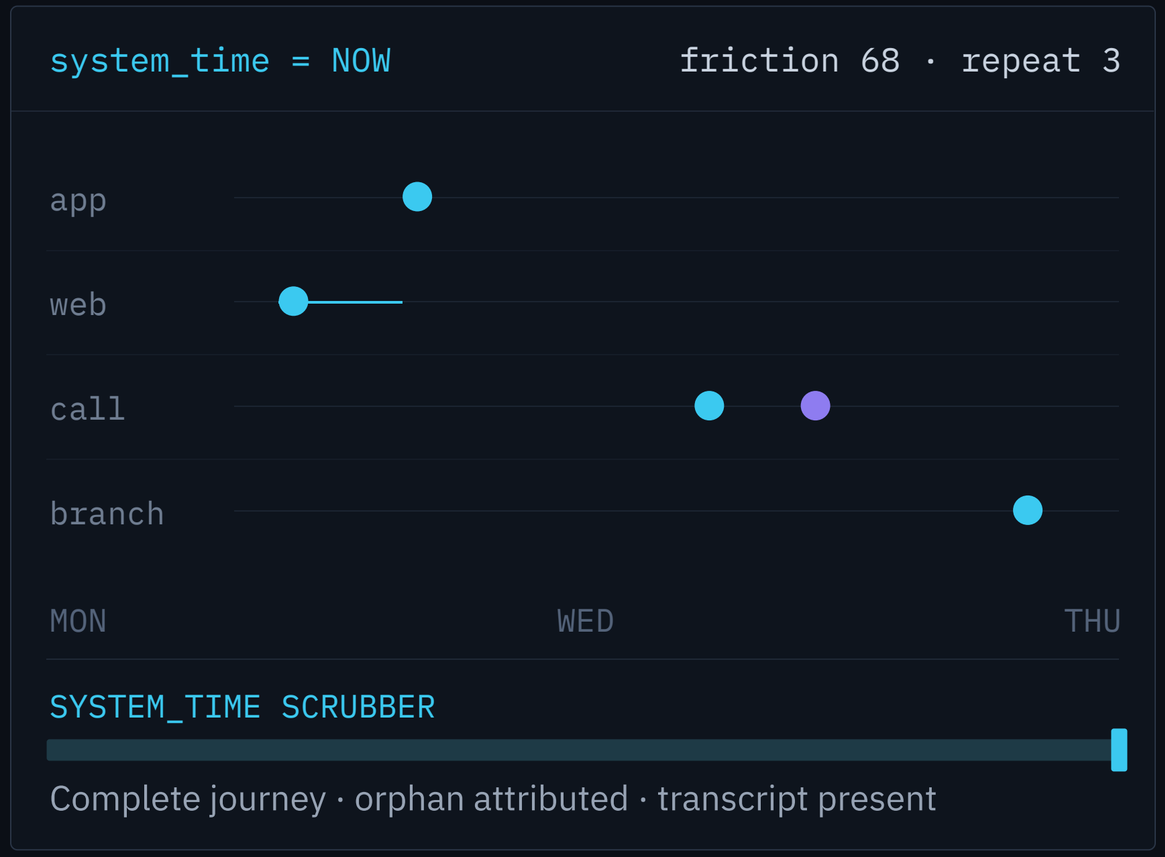
Console + hot path

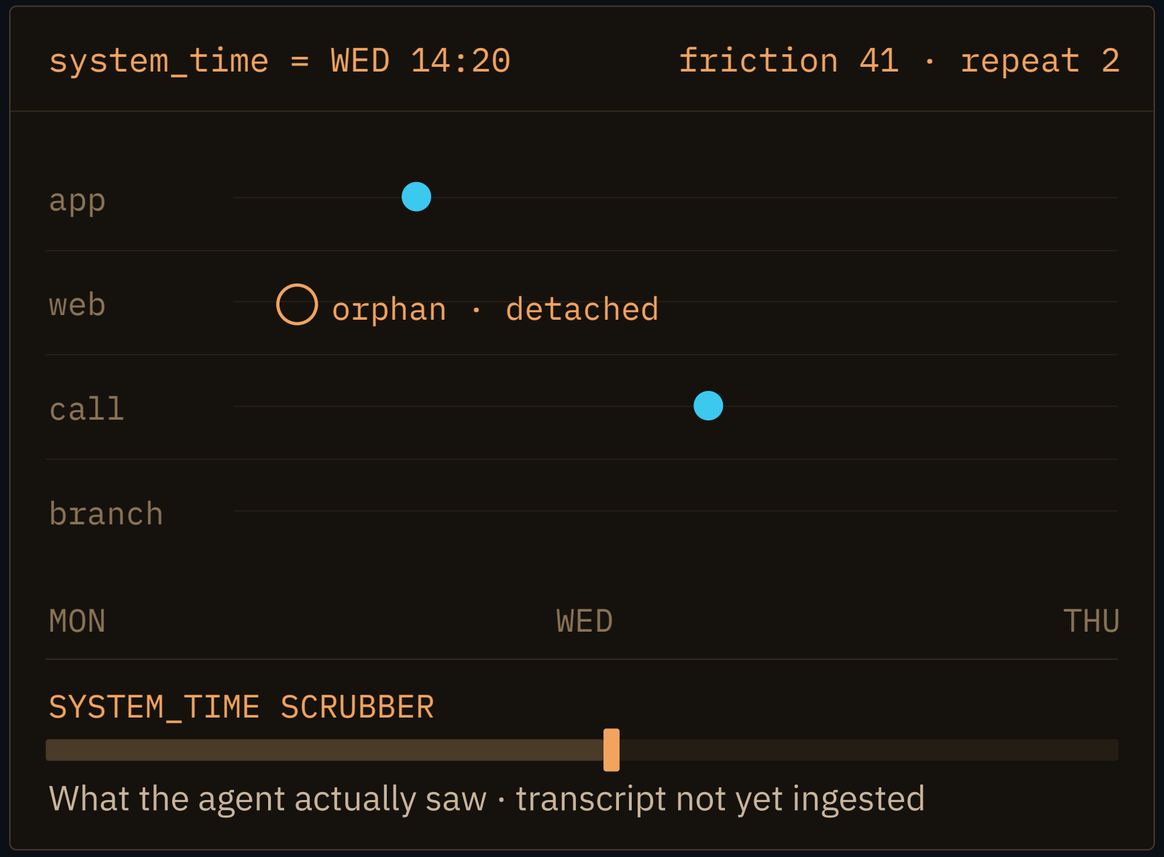
Redis 90-day window · FastAPI + SSE · React + visx with system_time scrubber

Every layer is independently replaceable with off-the-shelf components. OpenTelemetry traces end to end; Docker Compose, offline.

The Scrubber Replays Belief, Not Just Events

T1 · T2 · orphan





Same customer, same query, two positions of the scrubber. The difference is `knowledge_gap` = 2 events.

Companion views: Resolution Evidence Panel (per-field Fellegi-Sunter weights, auditable Unmerge with reason capture) and Knowledge Gap Report.

Ground Truth Is Hidden, So Accuracy Is Measured

Synthetic generator with hidden `true_customer_id`, interleaved intents, arrival delays, ~15% seeded broken handoffs.

CORRECTNESS

identity_f1

Pairwise precision / recall against hidden labels.

false_merge_rate < 0.5%

Two Card Members wrongly joined. Treated as a privacy incident, not a ranking error.

thread_f1

Cluster agreement against seeded intent labels.

NOVEL TO THIS DESIGN

retroactive_attribution_yield

Share of orphans later claimed by a binding event.

attribution_lag

Distribution of orphan-to-binding delay.

knowledge_gap_at_contact_time

Events invisible to the agent at the moment of contact. Only computable with an AS-OF store.

post_revision_alert_precision

Correctness of alerts after later revisions land.

ABLATION

repeat_contact_false_positive_rate

Measured with intent threading on versus off, all else fixed. The one number that isolates the threading contribution.

NO ROI CLAIM

We have no access to internal baselines, so we quantify system behaviour instead of asserting business impact.

Every Component Has a Named Enterprise Swap

LAYER	PROTOTYPE	PRODUCTION PATH
Transport	Redpanda	Apache Kafka
Stream proc	Bytewax	Apache Flink
Linkage	Splink on DuckDB	Splink on Spark
Identity graph	Union-find, NetworkX	Graph store + PG log
Embeddings	all-MiniLM-L6-v2	Same via ONNX Runtime
Journey store	PostgreSQL 16	XTDB / partitioned PG
Hot path	Redis, 90-day	Redis Cluster
Analytics	DuckDB	Snowflake / BigQuery
Deployment	Docker Compose	Kubernetes

ZERO MARGINAL COST

Fully offline · GPU-free · laptop-runnable · no paid APIs

Splink gives explainable Fellegi-Sunter weights, so every match decision is auditable without a model server.

SCALES THE WAY THE ENTERPRISE NEEDS

01 Resolution and threading workers are stateless; state lives in compacted topics and Postgres.

02 Splink blocking rules bound candidates at $O(n \cdot k)$, not $O(n^2)$.

03 Customer-hash and time partitioning; cold partitions age to object storage; retractions bounded by allowed lateness.

04 A new channel needs only a producer and its signals.

EVERY UNIFIED-CUSTOMER-VIEW TOOL

replays the customer's journey.

THIS SYSTEM

replays its own belief about that journey.

It is the only way to answer, with evidence, the question a servicing organisation actually asks after something goes wrong:

“Was the information there, and did we act on it?”

Cross-channel stitching treated as a distributed-state problem — late arrival, late binding, revision, invalidation — using mature, off-the-shelf components. Nothing here requires inventing an algorithm.

valid_time · system_time